

AER1217 Lab 2

Dejan Plavsic - 1012836442
Chris McGale - 1005303841
Mohsen Movahedi - 1005371517

Purpose

Various quadcopter controllers exist, but PD is by far the most popular for its simplicity. In this lab we tested its effectiveness in the safe-control-gym environment and extended the requirements to compare it to the theoretically optimal LQR and growing popularity of MPC.

Derivations

There are three primary tasks in calculating the position controller, which are as follows:

1. Computing the desired acceleration
2. Computing the desired thrust
3. Computing the desired attitude command

Beginning with the desired acceleration command, which is a combination of the acceleration feedback (a_{fb}), reference acceleration (a_{ref}), and the gravitational acceleration term (g_z), seen below as follows:

$$\mathbf{a}_{des} = \mathbf{a}_{fb} + \mathbf{a}_{ref} + g\mathbf{z}_W$$

The desired acceleration (a_{des}) represents the necessary acceleration needed to be produced by the motors in order to continue along the desired path, which in this case was a circle. a_{ref} is just the acceleration required to continue on the desired trajectory, assuming perfect positioning, and the g_z is from the gravitational force, facing downwards in the world coordinate axis system. a_{fb} represents the acceleration provided by the controller to counter the effects of any deviation from the path, which the quadcopter may have experienced. The focus of this report was on the use of a PD controller, which allows the acceleration to be calculated via the following formula:

$$\mathbf{a}_{fb} = -\mathbf{K}_{pos}(\mathbf{p} - \mathbf{p}_{ref}) - \mathbf{K}_{vel}(\mathbf{v} - \mathbf{v}_{ref})$$

With the desired acceleration, it is possible to calculate the desired thrust. The desired thrust is simply calculated by multiplying the acceleration magnitude by the mass of the system. The magnitude of the acceleration can be used as thrust only acting in the z-body-axis:

$$c_{cmd} = m \|\mathbf{a}_{des}\|.$$

Using the a_{des} and the cartesian coordinates for a circle, we get the desired attitudes (body axis):

$$\mathbf{z}_{B,des} = \frac{\mathbf{a}_{des}}{\|\mathbf{a}_{des}\|}$$

$$\mathbf{x}_{B,des} = \frac{\mathbf{y}_C \times \mathbf{z}_{B,des}}{\|\mathbf{y}_C \times \mathbf{z}_{B,des}\|}$$

$$\mathbf{y}_{B,des} = \mathbf{z}_{B,des} \times \mathbf{x}_{B,des}$$

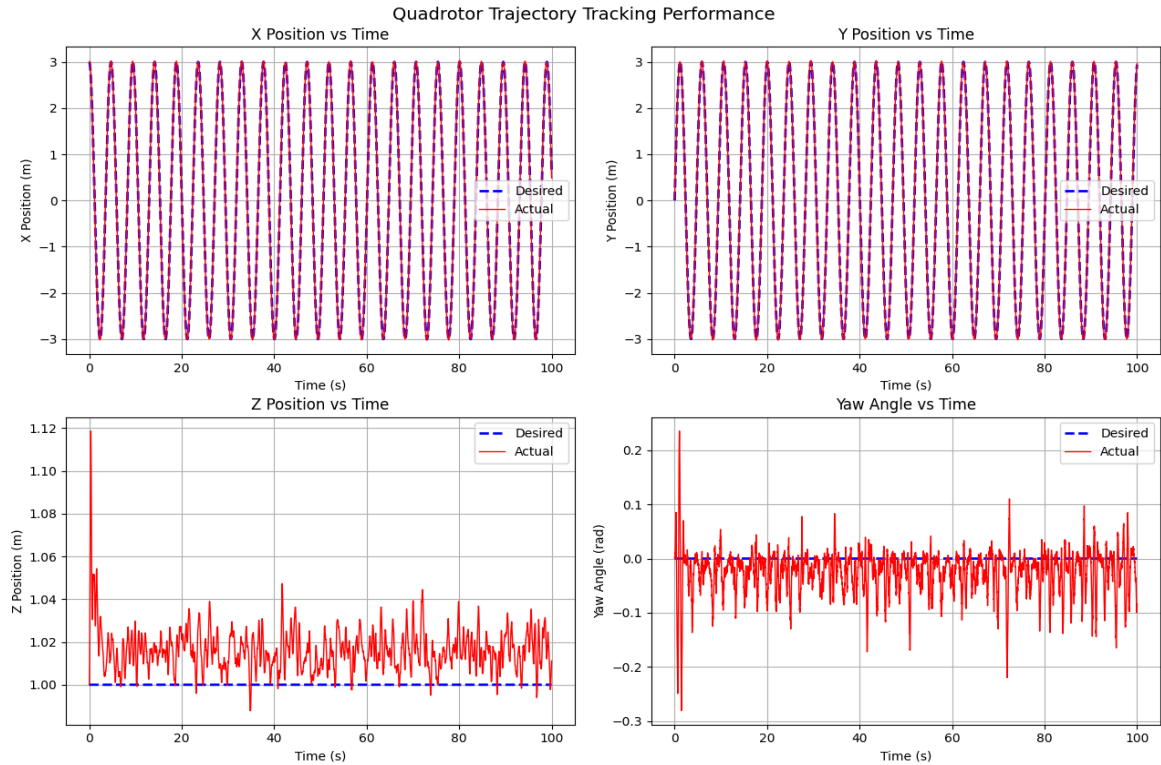
Starting the z-body-axis unit vector calculation, the acceleration must be co-linear with the z-body-axis, and so we normalize the acceleration vector to capture the unit vector of this axis. For the x-body-axis the process is slightly more involved including an additional term y_c . This term is simply the following vector showing the y-axis coordinated following the circular path, where phi is the yaw angle.

$$\mathbf{y}_C = [-\sin \psi \quad \cos \psi \quad 0]^T$$

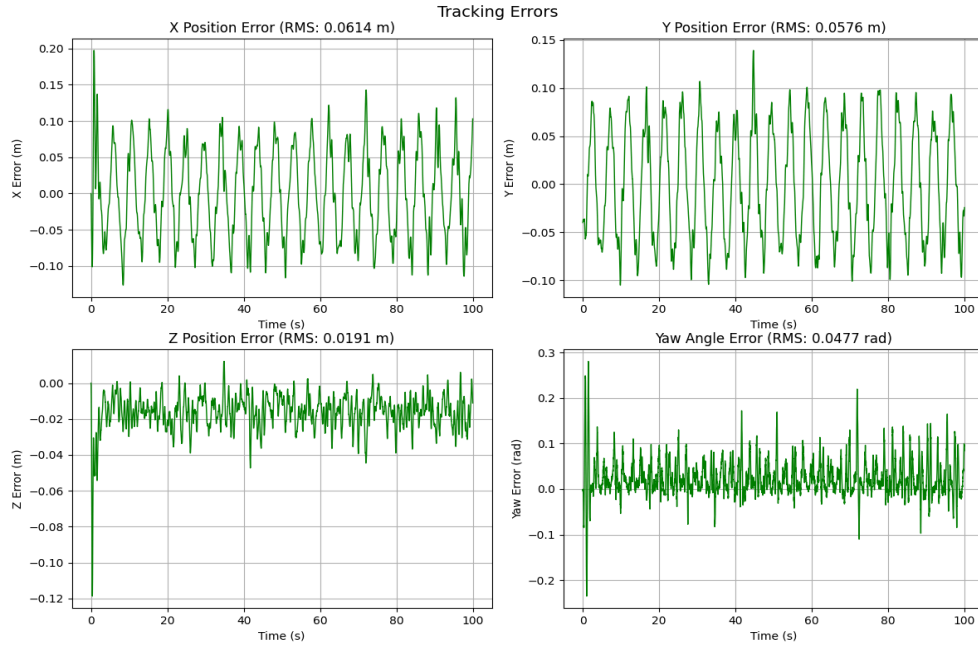
The x-body-axis must be normal to the z-body and y-body -axis. We can use the y_c value instead, which will be collinear, but have a varying magnitude from the y-body axis. Finding the y-body-axis is similar to finding the x-body axis, however instead of using an ' x_c ' value, it is possible to use the y-body-axis directly. Implementations of these functions can be found in `aer-course-project/controllers/base_controller.py`

Position and Attitude Graphs:

PD Controller - Desired & Actual in x, y, z, yaw:



PD Controller - Errors in x, y, z, yaw:



Tuning-Parameters & Controller Gains

We tuned gains using the following procedure: 1. Solve for hover condition (Z) to achieve. 2. Increased both x and y together until a stable response was found. 3. We kept a x,y:z ratio of about 1.3 as this was found to produce excellent vertical stability. To find the optimal gains we adjusted our main script to allow for a batch run of hundreds of gain combinations and reported on each. The minimum total rms found was ~0.26. Below are the final gain values chosen.

PD-controller:

- $K_{pos} = [8.0, 8.0, 10.0]$
- $K_{vel} = [2.0, 2.0, 2.0]$

The log RMS response for configurations that produced above 20 rotations in 100 seconds can be seen in Appendix A. According to the graph this seems to be near optimal

Results for PD controller:

Experiment SWEEP_Kp8_Kd2 had the best results with Average RMS: ~0.261

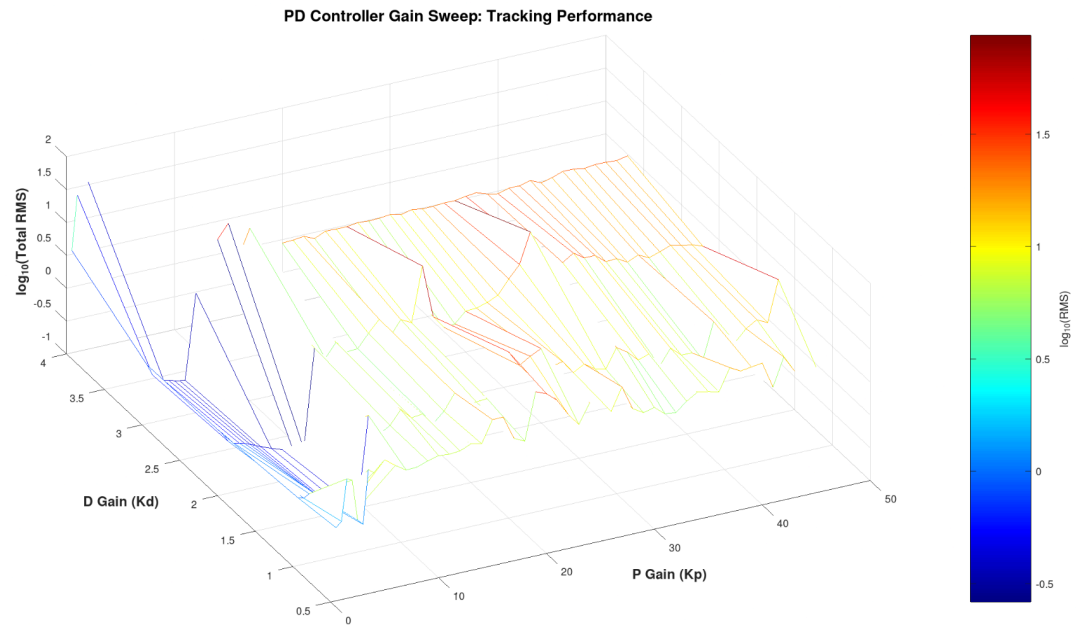
Consult the attached excel file for our /aer1217-project/aer-course-project/data/ folder in the command 4m/s sheet. The below discussions will show how far we pushed this result to an eventual **RMS of 0.0657**.

AI Acknowledgment

This project benefited from AI-assisted code organization and design critique. Claude was used to help clean up the structure of the codebase and provide feedback on architectural decisions, while all core ideas and implementation remain the work of the group members.

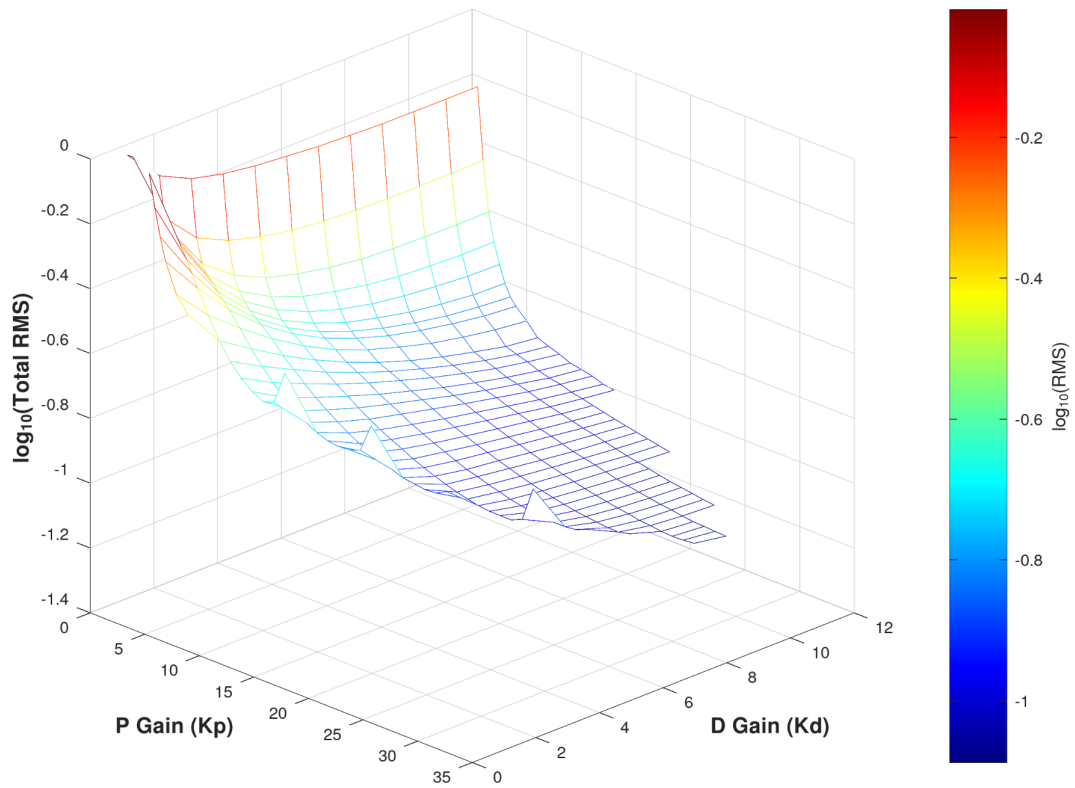
Appendix:

A. PID Controller Tuning Effects on RMS (Command 4m/s immediately)

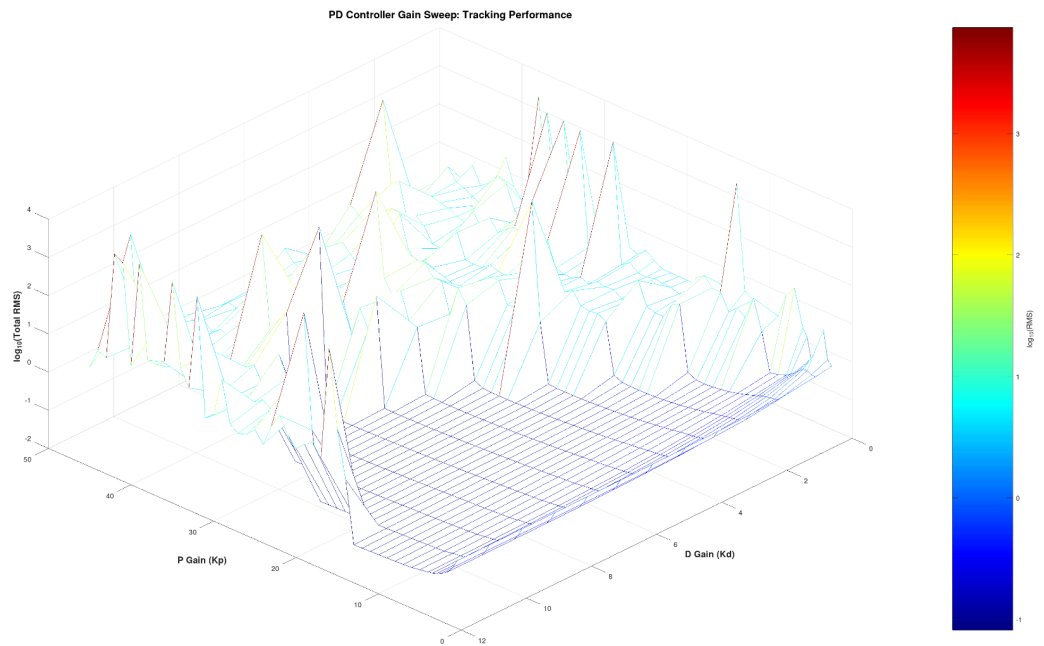


B. PID Controller Tuning Effects on RMS (No ramp, no climb)

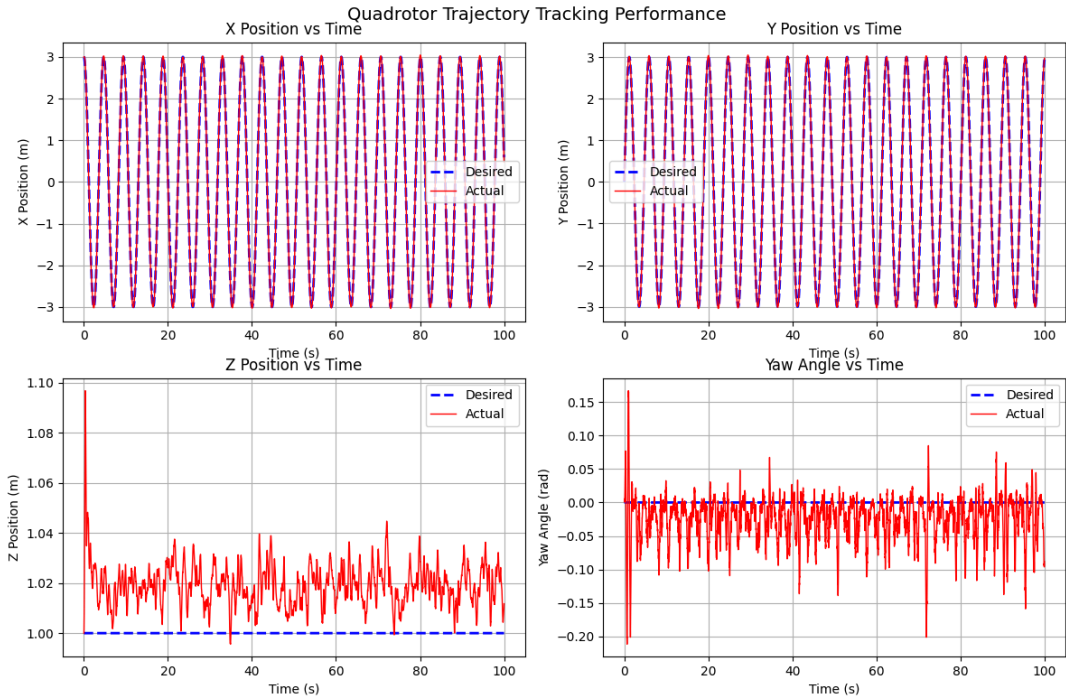
PD Controller Gain Sweep: Tracking Performance



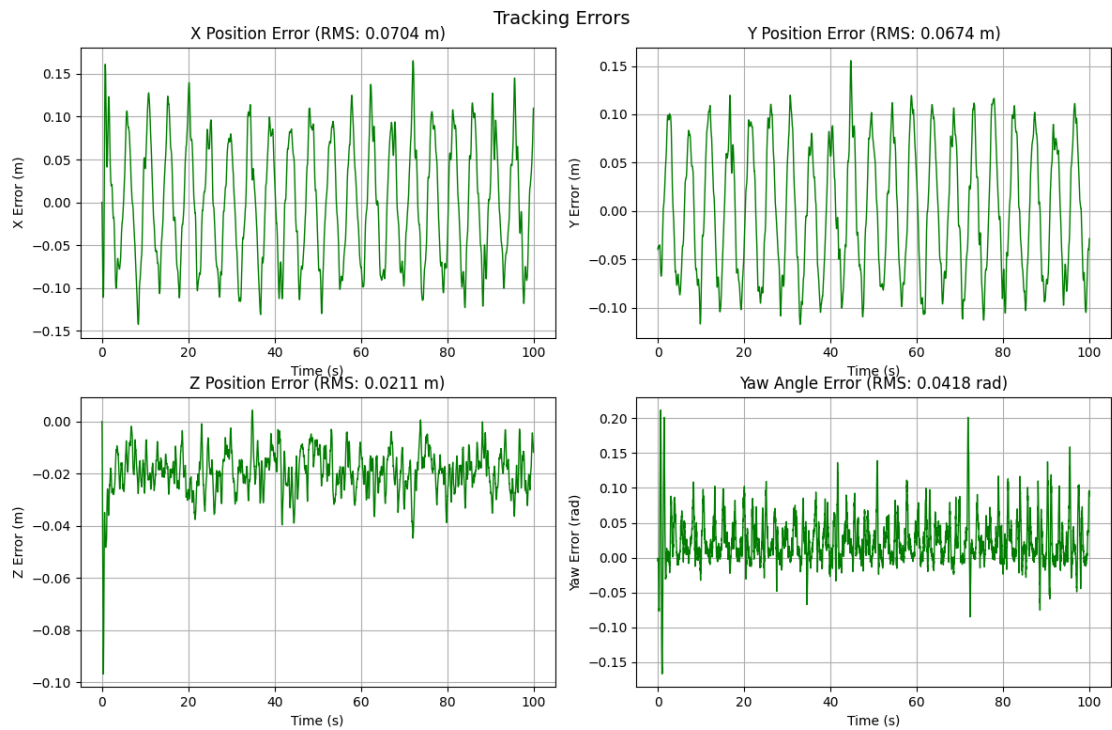
C. PID Controller Tuning Effects on RMS (No ramp, no climb) Unfiltered results



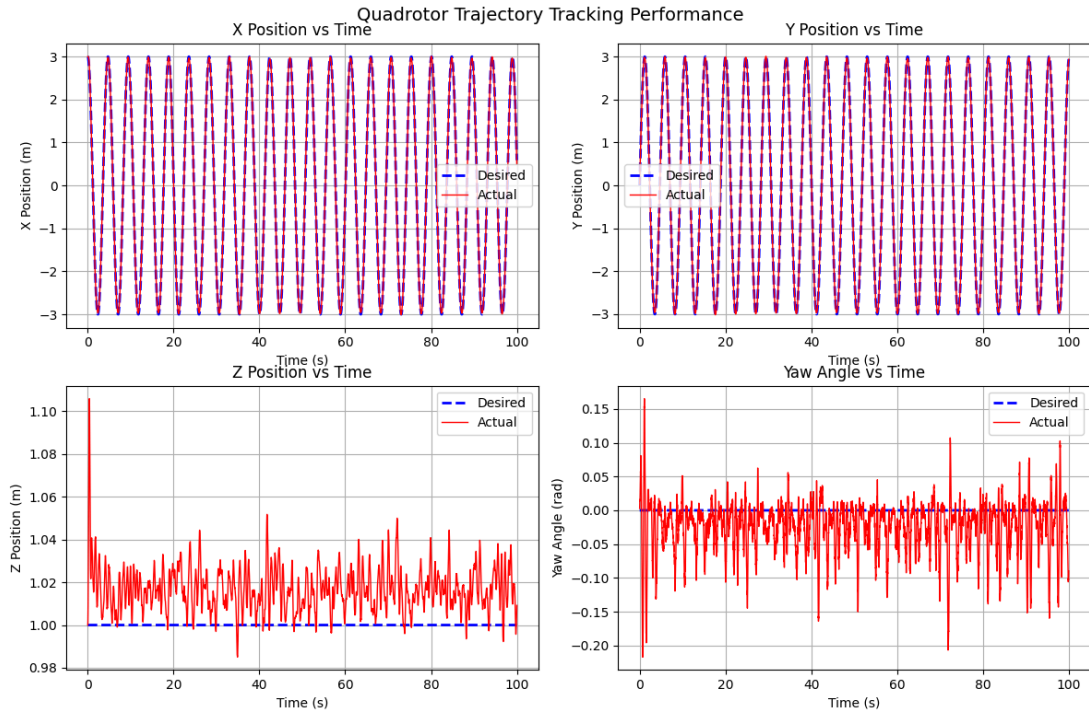
D. LQR Controller - Desired & Actual x, y, z, yaw:



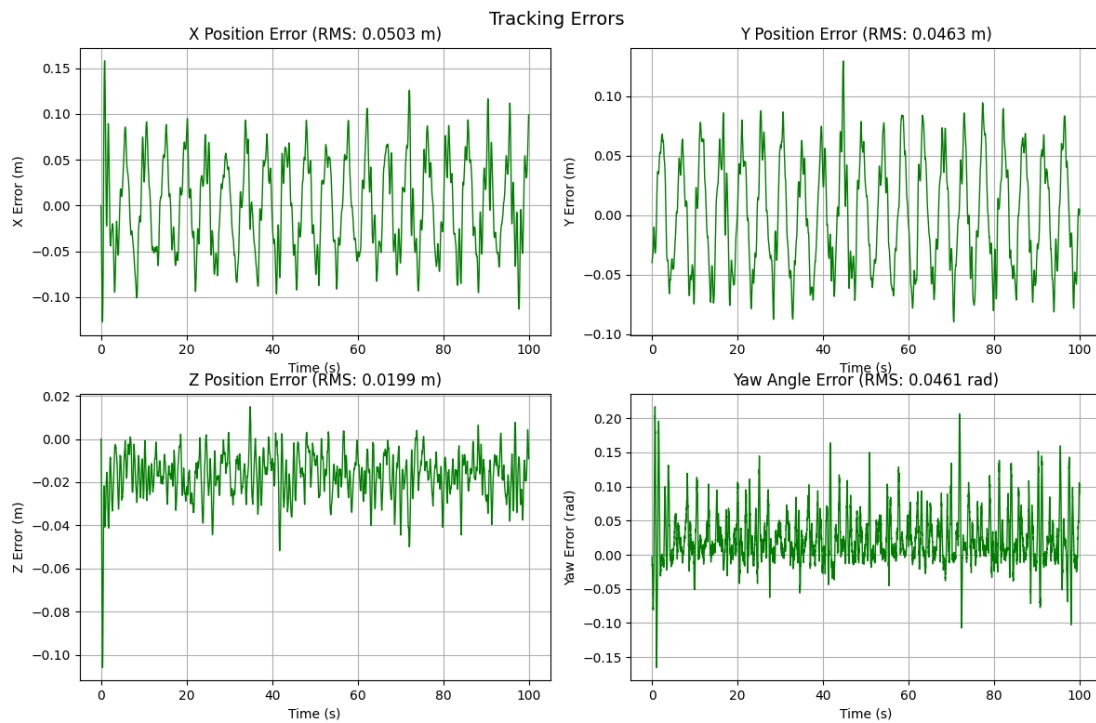
E. LQR Controller - Errors in x, y, z, yaw:



F. MPC Controller - Desired & Actual x, y, z, yaw:



G. MPC Controller - Errors in x, y, z, yaw:



Discussion on Additional Content

Batch run system

The batch mode (--batch flag) loads a YAML config file (--batch-config) containing a list of experiments under an experiments key. Each experiment entry is a standardized dictionary specifying a name, controller_type, and controller parameters (like speed, gains, etc.).

The run_batch function iterates through all experiment configs, calling run_single_experiment for each one with GUI and plots disabled. It catches errors per-experiment so failures don't halt the sweep. After all runs complete, it prints a summary table comparing each experiment's name, controller type, speed, RMS error, max error, and laps completed. If --output is provided, results are saved to a CSV via pandas.

Speed Ramp Up

The quadrotor starts from rest (zero velocity), but the circular trajectory reference demands immediate tangential speed. Without accounting for this mismatch, the controller sees a large velocity error at $t=0$ and commands aggressive corrections, causing the quad to fly off or oscillate before settling.

The ramp-up solves this by linearly increasing the reference angular velocity from 0 to $\omega_{\max}$ over a configurable ramp_duration (default 2 seconds). You cannot simply scale ω in $\cos(\omega * t)$ because during the ramp, ω varies with time, so the angle must be computed as the integral of the time-varying angular velocity:

During ramp ($0 \leq t < T_{\text{ramp}}$):

- $\omega(t) = \omega_{\max} * t / T_{\text{ramp}}$
- $\text{angle}(t) = \omega_{\max} * t^2 / (2 * T_{\text{ramp}})$ (integral of ω)
- $\alpha(t) = \omega_{\max} / T_{\text{ramp}}$ (constant angular acceleration)

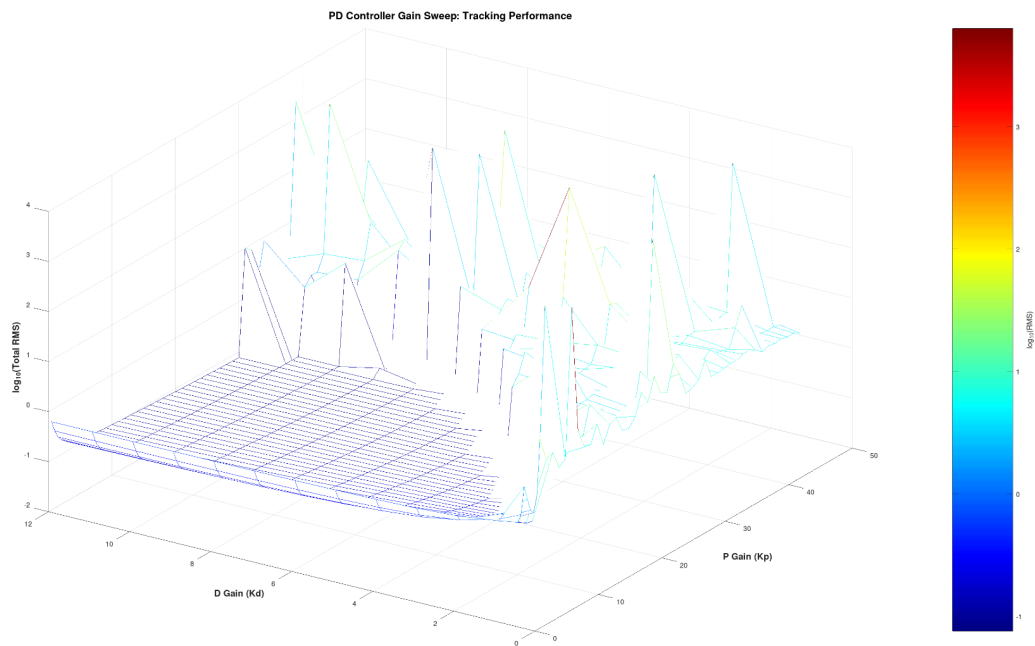
After ramp ($t \geq T_{\text{ramp}}$):

- $\omega(t) = \omega_{\max}$
- $\text{angle}(t) = \text{angle_at_ramp_end} + \omega_{\max} * (t - T_{\text{ramp}})$
- $\alpha(t) = 0$

The acceleration reference also picks up a tangential component ($r * \alpha$) during the ramp, in addition to the usual centripetal term ($r * \omega^2$). This gives the controller a physically consistent feedforward signal throughout the acceleration phase.

Setting ramp_duration: 0 in the config recovers the original instant-speed behavior, which is useful for comparison. The ramp duration is configurable per-experiment in both single and batch modes.

Results: Experiment SWEEP_Kp33_Kd11 resulted in a total rms of ~ 0.075 although it only produced 21.0 rotations compared to the 21.22 of the initial assignment.



Modular Controllers

We finished the PD controller early, so to further the learning opportunities of the lab, we reconfigured the project to allow for a modular controller and developed both a LQR and MPC controller which inherits from the base_controller sub class. These can be found in the controllers/ directory. PD/LQR overrides the base class `_get_gains()` while MPC controller sets accelerations directly via overwriting `compute_desired_acceleration`. Both used imported solvers.

LQR Controller

LQR stands for Linear Quadratic Regulator. "Linear" because it operates on a linearized system model, "quadratic" because it minimizes a quadratic cost function.

Q and **R** are the weight matrices in that cost function. **Q** (2×2) penalizes state error, controlling how much you care about position error vs velocity error. **R** (1×1) penalizes control effort, representing how "expensive" it is to apply acceleration. Larger **Q** entries demand tighter tracking; larger **R** encourages gentler inputs. Tuning the **Q/R** ratio is how you trade responsiveness against aggressiveness.

A and **B** define the linearized dynamics of a **double integrator**, the simplest model of position control where acceleration directly controls position through velocity.

The state vector is [position, velocity] (2 states), and the input is acceleration (1 input). So:

- **A is 2×2** because it maps the 2-element state to its derivative. The [0,1; 0,0] structure encodes "position derivative = velocity" and "velocity derivative = 0 (without input)."
- **B is 2×1** because it maps the 1 control input to the 2-element state derivative. The [0; 1] structure means acceleration only directly affects the velocity derivative.

The solver finds the optimal gain matrix K (1×2) via the Riccati equation, yielding one position gain and one velocity gain. These are then applied identically to x, y, and z axes, with an optional z-axis scaling factor since gravity makes the vertical axis behave differently.

Results:

With ramp a minimum RMS value of ~0.07 was produced, only slightly better than the PD value found a) The linearizations used in the LQR construction and b) The limited scope of Q and R testing compared to PD.

Name	Type	Speed	RMS (m)	Max (m)	Laps
------	------	-------	---------	---------	------

LQR_Qp150_Qv35_R0.25_z1.4 LQR	4.0	0.0791	0.1346	21.01	
-------------------------------	-----	--------	--------	-------	--

MPC Controller

MPC stands for Model Predictive Control. Instead of computing fixed gains once (like LQR), it solves an optimization problem at every timestep, finding the best sequence of control inputs over a finite future horizon.

Structurally, MPC differs from the other controllers because it bypasses the gain-based approach entirely. While PD and LQR implement `_get_gains()` to return position and velocity gain arrays, MPC returns None for both:

```
def _get_gains(self):
    """MPC doesn't use gains - return None."""
    return None, None
```

Instead, it overrides `compute_desired_acceleration()` directly, replacing the simple $K_p * \text{error} + K_d * \text{vel_error} + \text{feedforward}$ calculation with a full optimization solve at each timestep.

Q and R serve a similar role as in LQR. Q penalizes state tracking error (how far position and velocity deviate from the reference), and R penalizes control effort. Here they are diagonal matrices: Q is 6×6 (weighting all 6 states) and R is 3×3 (weighting the 3 acceleration inputs). There is also a terminal cost Q_N (Q scaled up by

10x) that more heavily penalizes error at the end of the prediction horizon to encourage convergence toward the reference.

A and B define *discrete-time* dynamics (unlike LQR which used continuous-time). The state is now 6-dimensional [x, y, z, vx, vy, vz] and the input is 3-dimensional [ax, ay, az]:

```
self.A = np.array([
    [1, 0, 0, dt, 0, 0 ],
    [0, 1, 0, 0, dt, 0 ],
    [0, 0, 1, 0, 0, dt],
    [0, 0, 0, 1, 0, 0 ],
    [0, 0, 0, 0, 1, 0 ],
    [0, 0, 0, 0, 0, 1 ],
])
self.B = np.array([
    [0.5*dt**2, 0, 0 ],
    [0, 0.5*dt**2, 0 ],
    [0, 0, 0.5*dt**2],
    [dt, 0, 0 ],
    [0, dt, 0 ],
    [0, 0, dt ],
])
```

A is 6×6 because it maps the full 6-element state to the next state. The upper-right dt entries encode $\text{position_next} = \text{position} + dt * \text{velocity}$, and the lower-right identity block gives $\text{velocity_next} = \text{velocity}$ (unchanged without input). It is the 3D version of the same double integrator from LQR.

B is 6×3 because it maps 3 acceleration inputs into the 6-element state. Each acceleration affects only its own axis: $\text{position} += 0.5*dt^2*a$ and $\text{velocity} += dt*a$. The block-diagonal structure means x-acceleration only touches x-position and x-velocity, and so on.

At each timestep, `compute_desired_acceleration` sets the current state as an initial condition, fills in the reference trajectory over the horizon, and solves a quadratic program (via `cvxpy/CLARABEL`) to find the optimal acceleration sequence. Only the first command is applied, then the whole process repeats the next step. This "receding horizon" approach lets MPC naturally handle constraints (acceleration and velocity limits) and adapt to changing references, at the cost of solving an optimization every control step. If the solver ever fails, it falls back to a simple PD controller:

```
def _fallback_control(self, cur_pos, cur_vel, target_pos, target_vel, target_acc):
    K_p = np.array([25.0, 25.0, 32.0])
    K_d = np.array([11.0, 11.0, 15.0])
    pos_e = target_pos - cur_pos
    vel_e = target_vel - cur_vel
    return K_p * pos_e + K_d * vel_e + target_acc
```

Results:

A decent improvement from ~ 0.075 $\rightarrow$ ~ 0.066 was achieved over the ramped PD. This was not perfect, but shows the promise of MPC and suggests that improvements could be made. Some ideas we have are: a) Control Rate Penalty (Δu smoothing) b) Better model selection and tuning.

```
=====
==
RESULTS SUMMARY
=====
==
```

Name	Type	Speed	RMS (m)	Max (m)	Laps
MPC_R0.2	MPC	4.0	0.0657	0.1322	21.01

For non ramped tests we found

MPC_Qpos10	MPC	4.0	0.1279	1.2954	21.22
------------	-----	-----	--------	--------	-------